MIESC

Multi-layer Intelligent Evaluation for Smart Contracts

Revert v4lend — Focused Triage

revert-finance/v4lend @ f6d0fb41 · src/ · 441 findings, 0 exploitable Highs · Cantina comp closed 03 Nov 2026 (publishable)

PUBLIC

Revert v4lend — MIESC audit triage memo

Target: `revert-finance/v4lend` @ `f6d0fb41` · scope `src/` (vault, oracle, hook, automators, transformers, shared math/planning/swap) **What it is:** A Uniswap v4-native lending & automation system. Lenders deposit one asset into a single-asset ERC4626 vault; borrowers post Uniswap v4 LP NFTs as collateral and borrow the vault asset. A v4 hook (`RevertHook`) runs on-swap automation (auto-range, auto-lend, auto-leverage, stop-loss/exit); standalone operator-driven automators do the same out-of-band; transformers run compounding/range/leverage flows via `V4Vault.transform`. Valuation is Chainlink-backed with pool/oracle deviation checks. Deploys on Arbitrum. **Toolchain:** Foundry / **solc 0.8.30** (solc-select venv, `forge --use`; `via_ir=true`, cancun) + **Slither 0.11.6**, reusing forge build-info. **Scan: 441 owned findings — 0 Critical / 36 High / 144 Medium / 163 Low / 98 Info**. **Method note:** MIESC ran Slither with its **default detector set**, deliberately *without* the project's `foundry.toml [lint] exclude_lints` (which hides `divide-before-multiply`, `erc20-unchecked-transfer`, `unsafe-typecast`, …), so those classes are surfaced and triaged.

> Triage layer. **Nothing here is a submission.** No PoC was built or submitted. The Cantina audit competition on this commit **closed 03 Nov 2026**, so this write-up carries no live-competition embargo; it reconciles MIESC's output against the project's own published Cantina Scan #1/#2 findings and audit handoff. Any disclosure is the user's own action.

Headline

All 36 High findings are false positives. The 36 break down as `reentrancy-eth` ×16, `reentrancy-balance` ×12, `uninitialized-state` ×4, `arbitrary-send-eth` ×3, `weak-prng` ×1 — every one accounted for below. The high-signal real-bug detectors (`incorrect-shift` ×3, `incorrect-exp` ×1) fired **only inside Uniswap v4-core dependencies** (`BitMath`, `TickBitmap`, `FullMath.mulDiv`) — out of scope, and canonical audited Uniswap code besides. Within first-party `src/`, the Highs reduce to

reentrancy under documented guards / trusted callees and ETH sends to the position's current NFT owner or trusted Uniswap routers. MIESC surfaced **nothing the two prior Cantina scans had not already found and fixed/accepted.** Sixth target in a row with 0 exploitable Highs.

> **Scoping correction worth recording:** an initial converter run reported 536 findings / 40 Highs. Normalizing paths revealed it was mis-attributing dependency files under `lib/.../src/…` (v4-core, v4-periphery, OpenZeppelin) as first-party because their paths *contain* `src/`. Fixing the owned-filter to "path **begins** with `src/`" (first-party files are `src/…`; deps are `lib/…`) dropped the count to 441 / 36 and removed all `incorrect-shift` / `incorrect-exp`. **A scanner that doesn't scope precisely manufactures its own false Highs** — a dogfooding lesson in its own right.

High-signal detectors — the ones that *can* be real bugs (all FP here)

`incorrect-shift` (3) + `incorrect-exp` (1) → out of scope (Uniswap v4-core)

All four map to `lib/uniswap-hooks/lib/v4-core/src/libraries/` — `BitMath.mostSignificantBit/leastSignificantBit`, `TickBitmap.flipTick`, and `FullMath.mulDiv`'s `(3 * denominator) ^ 2`. The last is the **canonical** Newton-iteration seed in Uniswap's FullMath (the `^` is intentional on an odd denominator, per the published derivation). Dependencies, not first-party; excluded after the scoping fix above.

`write-after-write` (2, Medium) → FP · `AutoLend.deposit` L182-194

`amount0/amount1` are assigned (L182-183), passed to `_quoteProtocolFees` whose returned `amount0/amount1` are then **overwritten unread** by `_netBalanceAfterReserved(currency, protocolFee)` (L193-194); only `protocolFee0/1` from the quote are used. `_quoteProtocolFees` is `internal pure` (confirmed), so discarding its amount returns loses no side effect. This is the **intentional whole-balance sweep model** the audit handoff flags ("whole-balance accounting … is intentional"): the contract settles from its *actual* balance net of reserved protocol fee, not from the quoted amount. Harmless; at most a redundant return value. Not exploitable. (This is a Medium, listed here with the other real-bug-class detectors.)

`uninitialized-state` (4, High) → FP · `RevertHookState` mappings

`_swapProtectionConfigs`, `_autoLendVaults`, `_lowerTriggerAfterSwap`, `_custodiedShares` are flagged "never initialized." They are **storage mappings written by the hook's action contracts that share the state spine** — verified writers exist: `_custodiedShares[vault] += shares` and `… = custodied > shares ? custodied - shares : 0` (`RevertHookAutoLendActions` L351/L508), `_autoLendVaults[token] = vault` (`RevertHookConfig`), `_swapProtectionConfigs[newTokenId] = …` (`RevertHookMigrationActions`); `_lowerTriggerAfterSwap[poolId]` is mutated via the `TickLinkedList` library acting on a storage reference. Slither's detector misses these cross-contract/by-reference writers in the documented shared-state-spine architecture. FP.

`arbitrary-send-eth` **(3) → FP**

- `Automator._transferToken` L146: raw `to.call{value}` is needed for native ETH, and every caller passes `to = _protocolFeeRecipient` or `recipient` (the position/NFT owner) — never an attacker-chosen address. Automator inherits OpenZeppelin `ReentrancyGuard`.
- `RevertHookSwapActions.modifyLiquiditiesWithPair` L42 and `Swapper._routerSwap` L78 send native value to the **trusted Uniswap** `PositionManager` **/** `UniversalRouter` — canonical protocol routers, not arbitrary recipients.

`weak-prng` **(1) → FP** · `HookLeaseController._chargeRent` **L977**

`state.feeCarry = uint16(feeUnits % BPS_DENOMINATOR)` is a **rent fee-carry remainder**, not randomness. FP.

The bulk of the Highs: reentrancy (verified per contract)

High-severity reentrancy = `reentrancy-eth` (16) + `reentrancy-balance` (12); `reentrancy-no-eth` (16) is Medium. They cluster in `V4Vault` (6), the `RevertHook*` action contracts, the automators, and the transformers. The gating differs by contract, so each family was checked rather than generalized:

- `V4Vault` (does *not* inherit a generic `ReentrancyGuard`): `transformedTokenId` is a **storage mutex** (L200, set L682, reset L742). `transform()` reverts if it is already set (L679), and `borrow()` / `liquidate()` revert while it is set (L836 / L910) — this blocks a transformer re-entering borrow/liquidate *during* a transform. For the **direct** path (a `liquidate` / `deposit` / `withdraw` / `repay` call not nested in a transform), the external calls are `safeTransfer(From)` of the single **governance-configured ERC20 vault asset** — a standard non-callback token — plus the trusted Uniswap `PositionManager`. No attacker-controlled reentrant callee exists on either path, so the FP holds without relying on a claim of strict CEI ordering. (Design documented at NatSpec L38-39 / L661.)
- **Automators**: inherit OpenZeppelin `ReentrancyGuard`; external calls are to the vault, the governance-configured ERC4626 lend vaults, and Uniswap routers.
- **Hook action contracts** (`RevertHookAutoLendActions`, 4 sites): the external calls are ERC4626 `deposit` / `redeem` on `state.autoLendVault` / `lendVault`, and `_custodiedShares` is updated *after* the call (what Slither flags). But the lend vault is **owner-registered** (`_autoLendVaults[token] = vault` in `RevertHookConfig`, governance only), so the callee is trusted, not attacker-chosen; `autoLendForceExit` L271 is additionally `msg.sender == owner` (current NFT owner) gated. FP — trusted callee, per-contract gating (not a uniform guard).

No attacker-controlled callee sits inside any of these flows; FP throughout — consistent with why the project does not chase `reentrancy-*` in its own lint.

Other classes (characterized)

Class	Count	Verdict
	4	**All** the canonical Uniswap tick-flooring idiom `(tick / tickSpacing) * tickSpacing` (`RevertHookTriggers._getTickLower`, `AutoLendLib` /

Class	Count	Verdict
`divide-before-multiply` (owned)		`AutoRangeLib.floorToSpacing`, `LeverageTransformer._getDummyTickRange`). The truncation **is** the intended grid-snap semantics. FP — exactly why the team excludes it from `foundry lint`.
`reentrancy-no-eth`	16 (Medium)	Same posture as the High reentrancy above — trusted callees (vault, configured ERC4626 vaults, Uniswap PositionManager), per-contract gating.
`incorrect-equality`	13	Benign zero/sentinel guards; characterized by detector expression, not individually opened.
`reentrancy-events` / `-benign`	51 / 21	Event-ordering / benign subclasses of the reentrancy posture above (Low/Info).
`unused-return` (82), `assembly` (21), `shadowing-local` (36), `timestamp` (39), `unused-state` (21)	—	Noise / standard; characterized by detector class, not individually opened.

Reconciliation against the project's known-issues baseline

The repo ships its audit trail in `docs/`: **Cantina Scan #1** (follow-up fixes V4LE-1/11/21/23/37/51/57/63/71/72/77 + deferred/accepted decisions) and **Cantina Scan #2 (2026-09-30)** — 37 findings (3 High: V4LE-97/98/156; 27 Medium; 3 Low; 4 Info), almost all **Fixed**, with V4LE-121 **Rejected** (the v4 `toInt128` settlement-bound window is unreachable) and documented accepted tradeoffs (`AUDIT-ACCEPTED-HOOK-SWAP-NO-SLIPPAGE-FLOOR`, non-vault remint automation loss, whole-balance sweep). For this triage I read those finding tables and `docs/audit-handoff.md` in full; I did **not** re-derive the prior auditors' PoCs. Every MIESC finding resolved to a false positive — and a false positive cannot be a novel real finding — so no reconciliation conflict arises. What the baseline confirms is that the exact areas MIESC's detectors touched (tick math, whole-balance accounting, the vault's `transformedTokenId` reentrancy model, hook swap routing) are already reasoned and, where relevant, fixed or explicitly accepted.

Bottom line

- **No submittable bug** found by static scan + triage — **sixth** target in a row (Lombard, Doppler, Valantis STEX, YieldNest, Arcadia, **v4lend**) with all raw Highs being false positives.
- Two findings of the run were about **the scanner, not the protocol**: (1) default-detector Slither fires `incorrect-shift` / `incorrect-exp` on canonical Uniswap v4-core libraries, and (2) imprecise path scoping silently promotes dependency findings to first-party Highs. Both are triage-layer lessons MIESC should encode, not protocol defects.
- Honest limits: verdicts are from code reading against the contract NatSpec, `docs/audit-handoff.md`, and the Cantina Scan #1/#2 tables; **no PoC was built or run** (a runnable PoC is mandatory for any real submission). MIESC's Slither lens does not cover the deep semantic/accounting classes the Cantina competition targeted.

Appendix — Evidence trail

- Normalized scan: `results.json` (owned = first-party `src/`; dependency paths under `lib/` excluded).
- Reports: `triage-focused.pdf` (focused triage); the full premium PDF is regenerable from `results.json` and is not committed.
- Toolchain: HTTPS clone @ `f6d0fb41` + recursive submodules → `forge build --root <repo> --use <solc-0.8.30> --build-info --skip test/script` → `slither . --foundry-ignore-compile --json` (slither 0.11.6, default detectors).
- Target pin: `revert-finance/v4lend` @ `f6d0fb41b7487f60ce64e011312a956e184e2a75`.
- Only **public source from the public GitHub repository** was accessed; **static analysis only**, no live systems / RPC / credentials, no exploitation, nothing submitted anywhere.